

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 1

Course: Operating Systems

Topic: Track A: The OS as a Resource Manager

Done by:

st. gr. FAF-243, Ilie Covali

Checked by:

Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 1 · Track A: The OS as a Resource Manager

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 1 (Track A: The OS as a Resource Manager, Linux)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.2.8-1-cachyos)
- **Terminal:** Alacritty / Bash
- **Workspace:** `/home/ilie/Documents/SO/os-lab1` (`~/os-lab1`)

Purpose of the Work

1. Study the role of the operating system as a primary hardware resource manager coordinating storage, processes, memory, and devices.
2. Inspect the Unix directory tree, file metadata, ownership, and Discretionary Access Control (DAC) permissions using standard core utilities.
3. Investigate the process lifecycle, job control, process states, and internal kernel process descriptors via the `/proc` virtual pseudo-filesystem.
4. Analyze physical memory utilization, swap configuration, buffer/cache allocations, and block storage devices.

System Identification

```
❯ whoami
ilie
❯ uname -a
Linux thinkbook 7.2.8-1-cachyos #1 SMP PREEMPT_DYNAMIC Sat, 26 Sep 2026 19:43:45 +0000 x86_64 GNU/Linux
❯ uptime
 16:09:20 up 23:50,  1 user,  load average: 3.28, 2.76, 2.18
```

Figure 0.1 – System identification, kernel version, and uptime

---

Part 1. Files and directories (File management)

1. Navigation and Inspection

```
❯ pwd
/home/ilie/Documents/SO/os-lab1
❯ ls -la /
total 20
dr-xr-xr-x   1 root root  160 Aug 16 14:06 .
dr-xr-xr-x   1 root root  160 Aug 16 14:06 ..
lrwxrwxrwx   1 root root    7 Oct 12  2025 bin -> usr/bin
drwx------   5 root root 4096 Jan  1  1970 boot
drwxr-xr-x  20 root root 4360 Sep 27 16:18 dev
drwxr-xr-x   1 root root 4576 Sep 28 15:43 etc
drwxr-xr-x   1 root root   64 Aug  2 20:55 Games
drwxr-xr-x   1 root root    8 Jun 18 12:49 home
lrwxrwxrwx   1 root root    7 Oct 12  2025 lib -> usr/lib
lrwxrwxrwx   1 root root    7 Oct 12  2025 lib64 -> usr/lib
drwxr-xr-x   1 root root   14 Sep 13 23:30 mnt
drwxr-xr-x   1 root root  340 Sep 22 11:20 opt
drwxr-xr-x   1 root root    4 Aug 16 14:06 path
dr-xr-xr-x 389 root root    0 Sep 27 16:18 proc
drwxr-x---   1 root root  158 Aug 16 12:26 root
```

```
drwxr-xr-x  45 root root 1100 Sep 28 16:00 run
lrwxrwxrwx   1 root root    7 Oct 12  2025 sbin -> usr/bin
drwxr-x---   1 root root  302 Sep 28 15:48 .snapshots
drwxr-xr-x   1 root root   24 Aug 16 14:49 srv
dr-xr-xr-x  13 root root    0 Sep 27 16:18 sys
drwxrwxrwt  51 root root 1580 Sep 28 16:11 tmp
drwxr-xr-x   1 root root   94 Sep 28 15:43 usr
drwxr-xr-x   1 root root  152 Sep 27 16:18 var
❯ ls -la ~
total 22408
drwx------ 1 ilie ilie     1292 Sep 28 16:11 .
drwxr-xr-x 1 root root        8 Jun 18 12:49 ..
drwxr-x--- 1 ilie ilie       84 Sep 21 15:28 .android
drwxr-xr-x 1 ilie ilie       90 Jul 18 21:37 .ansible
drwxr-xr-x 1 ilie ilie       38 Jul 29 12:29 .antigravity-ide
drwxr-xr-x 1 ilie ilie      314 Sep  3 11:05 .anydesk
drwxr-x--- 1 ilie ilie        8 Aug 15 00:31 .atuin
-rw------- 1 ilie ilie      436 Aug  4 23:08 .bash_history
-rw-r--r-- 1 ilie ilie       21 Jun 18 12:49 .bash_logout
-rw-r--r-- 1 ilie ilie       57 Jun 18 12:49 .bash_profile
-rw-r--r-- 1 ilie ilie      202 Sep 21 15:20 .bashrc
drwxr-xr-x 1 ilie ilie       10 Sep 21 15:20 bin
-rwx------ 1 ilie ilie 22228992 Jun 21 12:14 blockchain.db
drwxr-xr-x 1 ilie ilie      806 Sep 27 21:46 Bottles
drwxr-xr-x 1 ilie ilie       14 Jul 18 12:50 .bun
drwx------ 1 ilie ilie     2362 Sep 28 16:09 .cache
drwxr-xr-x 1 ilie ilie      118 Aug  5 10:41 .cargo
drwxr-xr-x 1 ilie ilie     5006 Sep 28 15:41 .config
drwx------ 1 ilie ilie       36 Aug 13 12:48 .copilot
drwxr-xr-x 1 ilie ilie      316 Sep 27 21:27 Desktop
drwx------ 1 ilie ilie       66 Sep 14 12:20 .docker
drwxr-xr-x 1 ilie ilie      344 Sep 28 16:08 Documents
drwxr-xr-x 1 ilie ilie       12 Aug 10 18:22 .dotnet
drwxr-xr-x 1 ilie ilie     4386 Sep 28 15:55 Downloads
drwxr-xr-x 1 ilie ilie       10 Jun 18 15:52 .factorio
drwxr-xr-x 1 ilie ilie        6 Sep 27 19:17 Games
drwxr-xr-x 1 ilie ilie       42 Jul 29 12:29 .gemini
-rw-r--r-- 1 ilie ilie       80 Aug 18 17:02 .gitconfig
drwxr-xr-x 1 ilie ilie       30 Aug 20 12:15 .gitlab
drwx------ 1 ilie ilie      116 Jun 18 12:58 .gnupg
drwxr-xr-x 1 ilie ilie        6 Aug  5 10:40 go
-rw-r--r-- 1 ilie ilie      355 Sep 27 22:15 .gtkrc-2.0
drwxr-xr-x 1 ilie ilie      214 Jul 29 12:27 .kimi
drwx------ 1 ilie ilie       42 Aug 18 14:37 .kube
drwxr-xr-x 1 ilie ilie       10 Aug  5 12:39 .lemminx
drwxr-xr-x 1 ilie ilie       26 Aug  4 23:01 .local
drwxr-xr-x 1 ilie ilie       34 Aug  5 09:29 .m2
drwx------ 1 ilie ilie       44 Jun 22 00:07 .mozilla
drwxr-xr-x 1 ilie ilie        0 Jun 18 12:52 Music
drwxr-xr-x 1 ilie ilie       92 Sep 14 11:46 .npm
drwxr-xr-x 1 ilie ilie       66 Aug  5 10:49 .nvm
-rw-r--r-- 1 ilie ilie     9886 Jul  8 22:32 .p10k.zsh
drwxr-xr-x 1 ilie ilie       36 Aug 25 14:37 Pictures
drwx------ 1 ilie ilie       10 Jun 18 13:04 .pki
drwxr-xr-x 1 ilie ilie      122 Sep 27 16:35 Projects
drwxr-xr-x 1 ilie ilie        0 Jun 18 12:52 Public
-rw------- 1 ilie ilie      256 Jun 18 14:28 .pulse-cookie
-rw-r--r-- 1 ilie ilie        0 Aug  4 23:04 .python_history
drwxr-xr-x 1 ilie ilie        6 Aug  9 11:47 .rancher
drwxr-xr-x 1 ilie ilie       78 Aug 14 15:13 .renpy
-rw-r--r-- 1 ilie ilie      317 Sep 28 14:00 sodomized.cow
drwxr-xr-x 1 ilie ilie      186 Sep  9 16:47 .ssh
drwxr-xr-x 1 ilie ilie      190 Sep 17 12:06 .steam
lrwxrwxrwx 1 ilie ilie       29 Sep 14 02:03 .steampath -> /home/ilie/.steam/sdk32/steam
lrwxrwxrwx 1 ilie ilie       27 Sep 14 02:03 .steampid -> /home/ilie/.steam/steam.pid
```

```
drwxr-xr-x 1 ilie ilie       24 Aug 10 18:23 .sts4
drwxr-xr-x 1 ilie ilie        0 Jun 18 12:52 Templates
drwxr-xr-x 1 ilie ilie       72 Jul 22 15:53 .terraform.d
drwxr-xr-x 1 ilie ilie        6 Jun 18 12:52 .var
-rw------- 1 ilie ilie       10 Jul 19 20:35 .vault_pass
drwxr-xr-x 1 ilie ilie       14 Jul 23 11:27 Videos
-rw------- 1 ilie ilie    16189 Jul 22 16:22 .viminfo
drwxr-xr-x 1 ilie ilie       44 Aug 10 18:20 .vscode
drwxr-xr-x 1 ilie ilie       26 Aug 10 18:20 .vscode-shared
-rw-r--r-- 1 ilie ilie      219 Aug 16 13:24 .wget-hsts
drwxr-xr-x 1 ilie ilie      126 Sep 27 16:45 .wine
-rw-r--r-- 1 ilie ilie       50 Aug 24 01:10 .Xresources
-rw-r--r-- 1 ilie ilie    57149 Jul 14 02:06 .zcompdump-thinkbook-5.9.1
-r--r--r-- 1 ilie ilie   123024 Jul 14 02:06 .zcompdump-thinkbook-5.9.1.zwc
-rw-r--r-- 1 ilie ilie    57469 Sep 27 19:21 .zcompdump-thinkbook-5.9.2
-r--r--r-- 1 ilie ilie   124872 Sep 27 19:21 .zcompdump-thinkbook-5.9.2.zwc
-rw-r--r-- 1 ilie ilie       47 Aug  4 23:02 .zshenv
-rw------- 1 ilie ilie   251856 Sep 28 16:11 .zsh_history
-rw-r--r-- 1 ilie ilie      706 Aug 15 01:41 .zshrc
❯ cd /etc && ls | head
adjtime
alfis.toml
alsa
ananicy.d
arch-release
arptables.conf
audisp
audit
avahi
bash.bash_logout
```

Figure 1.1 – Navigation and root directory inspection

2. Create, Copy, Rename, and Remove

```
❯ cd /home/ilie/Documents/SO/os-lab1
❯ mkdir -p demo && cd demo
❯ echo "hello operating systems" > note.txt
❯ cp note.txt copy.txt
❯ mv copy.txt renamed.txt
❯ ls -l
total 8
-rw-r--r-- 1 ilie ilie 24 Sep 28 16:22 note.txt
-rw-r--r-- 1 ilie ilie 24 Sep 28 16:22 renamed.txt
❯ rm renamed.txt
❯ ls -l
total 4
-rw-r--r-- 1 ilie ilie 24 Sep 28 16:22 note.txt
```

Figure 1.2 – File manipulation lifecycle in demo directory

3. Ownership and Permissions

```
❯ ls -l note.txt
-rw-r--r-- 1 ilie ilie 24 Sep 28 16:22 note.txt
❯ chmod 600 note.txt
❯ ls -l note.txt
-rw------- 1 ilie ilie 24 Sep 28 16:22 note.txt
❯ chmod 644 note.txt
❯ ls -l note.txt
-rw-r--r-- 1 ilie ilie 24 Sep 28 16:22 note.txt
```

Figure 1.3 – Modifying file permissions with chmod

Observe (Part 1 Answers)

1. **Who owns the files you just created, and which group?**
   - **Owner:** `ilie` (user ID `1000`).
   - **Group:** `ilie` (group ID `1000`).
   - The OS automatically assigned the credentials of the active user session as the owner and primary group of the newly created files.
2. **What do the ten characters at the start of `ls -l` mean for `note.txt` after `chmod 600`?**
   - The string is `-rw-------`.
     - `1st character (-)`: File type indicator, showing this is a standard regular file (not a directory `d` or symbolic link `l`).
     - `Characters 2–4 (rw-)`: User/Owner permissions. Read (`r`) and write (`w`) permissions are granted, while execute (`-`) is denied.
     - `Characters 5–7 (---)`: Group permissions. Read, write, and execute permissions are all denied.
     - `Characters 8–10 (---)`: Others/World permissions. Read, write, and execute permissions are all denied.
   - In octal, `600` represents binary `110 000 000`, restricting access exclusively to the owner.
3. **Name two directories directly under `/` and say, in one line each, what they hold.**
   - `/etc`: Contains system-wide configuration files and initialization scripts for installed packages and services.
   - `/proc`: A virtual pseudo-filesystem synthesized dynamically by the Linux kernel that exposes runtime process and kernel hardware state.

Interesting Output Highlight (Part 1)

Inspecting `ls -la /` reveals that virtual filesystem mount points like `/proc` and `/sys` report a size of `0` bytes. Unlike real directories on the storage drive, they consume no disk space because they are dynamically populated in-memory by the kernel on demand.

Part 2. Processes (Process management)

4. Process Listing

```
❯ ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0  23292 15712 ?        Ss   Sep27   0:19 /usr/lib/systemd/systemd --switched-root
↪  --system --deserialize=60 splash
root           2  0.0  0.0      0     0 ?        S    Sep27   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    Sep27   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   Sep27   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   Sep27   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   Sep27   0:00 [kworker/R-kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   Sep27   0:00 [kworker/R-slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   Sep27   0:00 [kworker/R-netns]
root          11  0.0  0.0      0     0 ?        I<   Sep27   0:00 [kworker/0:0H-kblockd]

❯ ps aux | wc -l
377
```

Figure 2.1 – Active process tree inspection with ps

5. Managing Background Jobs

```
❯ sleep 300 &
[1] 599116

❯ jobs
[1]+  Running                 sleep 300 &

❯ ps -ef | grep sleep
```

```
ilie      599116  584870  0 16:22 pts/5    00:00:00 sleep 300
ilie      599118  584870  0 16:22 pts/5    00:00:00 grep sleep

❯ kill 599116
[1]+  Terminated              sleep 300
```

Figure 2.2 – Background job control with jobs, bg, and kill

6. Process Introspection via `/proc`

```
❯ sleep 300 &
[1] 599119

❯ PID=599119

❯ ls /proc/$PID/
arch_status         fdinfo              ns              smaps_rollup
attr                gid_map             numa_maps       stack
autogroup           io                  oom_adj         stat
auxv                ksm_merging_pages   oom_score       statm
cgroup              ksm_stat            oom_score_adj   status
clear_refs          limits              pagemap         syscall
cmdline             loginuid            personality     task
comm                map_files           projid_map      timens_offsets
coredump_filter     maps                root            timers
cpu_resctrl_groups  mem                 sched           timerslack_ns
cwd                 mountinfo           schedstat       uid_map
environ             mounts              sessionid       wchan
exe                 mountstats          setgroups
fd                  net                 smaps

❯ cat /proc/$PID/status | head -20
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   599119
Ngid:   0
Pid:    599119
PPid:   584870
TracerPid:  0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 3 90 954 959 980 982 983 985 989 992 995 998 1000
NStgid: 599119
NSpid:  599119
NSpgid: 584870
NSsid:  584870
Kthread:    0
VmPeak:     6312 kB
VmSize:     6312 kB
VmLck:         0 kB

❯ kill $PID
```

Figure 2.3 – Process metadata and status in /proc/<pid>/

Observe (Part 2 Answers)

1. **What is the PID of process number 1, and what is it (look it up)?**
   - **PID:** 1.
   - **Program:** `systemd` (`/usr/lib/systemd/systemd`).
   - It is the init system and service manager for Linux, acting as the very first user-space process started by the Linux kernel after booting; all other user processes and background service daemons are direct or indirect descendants of process 1.

2. **Roughly how many processes were running on your idle VM?**
   - About `375` processes (`ps aux | wc -l`). A large portion of these are internal kernel threads (such as `[kthreadd]`, `[kworker/...]`, and interrupt handlers) that assist the OS in managing hardware, alongside systemd background daemons and desktop environment tasks.
3. **From `/proc/<pid>/status`, what does the "State:" line say for your sleeping process?**
   - `State: S (sleeping)`. This designates an interruptible sleep state, indicating the process is idle and waiting on an OS timer event (the expiration of its 300-second sleep interval) without consuming CPU cycles.

Interesting Output Highlight (Part 2)

The contents of `/proc/$PID/` showcase how the OS kernel exposes internal process control blocks (PCBs) as standard directories. Key entries like `exe` (symlink to the binary), `fd/` (directory of active file descriptor handles), and `maps` (virtual address space layout) allow direct inspection of a running program without attached debuggers.

Part 3. Memory (Memory management)

Memory Inspection

```
❯ free -h
               total        used        free      shared  buff/cache   available
Mem:            19Gi       9.3Gi       5.8Gi       191Mi       4.9Gi        10Gi
Swap:           19Gi       2.1Gi        17Gi

❯ cat /proc/meminfo | head -6
MemTotal:       20338228 kB
MemFree:         6081672 kB
MemAvailable:   10608164 kB
Buffers:            2248 kB
Cached:          4431108 kB
SwapCached:          712 kB

❯ sleep 300 & PID=$!
[1] 599201

❯ grep VmRSS /proc/$PID/status
VmRSS:      1816 kB

❯ kill $PID
```

Figure 3.1 – Physical memory and swap allocation breakdown

Observe (Part 3 Answers)

1. **How much total RAM does the VM have, and how much is free?**
   - **Total RAM:** Approximately `19 GiB` (`20,338,228 kB` ~ 20.3 GB).
   - **Free RAM:** `5.8 GiB` strictly unallocated (`MemFree`), while `10 GiB` is reported as `available` (`MemAvailable`). The `available` value reflects actual usable memory because the OS can immediately evict reclaimable page caches and file buffers (`buff/cache: ~4.9 GiB`) if application programs request memory.
2. **What is swap, and how much is configured?**
   - **What swap is:** Swap is an overflow area (either a dedicated disk partition, file, or compressed RAM segment) utilized by the OS virtual memory manager to page out inactive memory pages, freeing physical RAM for active working sets and caching.
   - **Configured swap:** `19 GiB` (specifically `19.4 GiB` configured via `/dev/zram0`, an in-RAM compressed block device), with `2.1 GiB` used and `17 GiB` free.
3. **How much resident memory (VmRSS) does a bare `sleep` process use, and does that surprise you?**
   - **Resident memory:** `1816 kB` (~1.8 MB).

- **Reflection:** While 1.8 MB initially seems large for a program that merely pauses execution, it is expected under modern Linux. The resident memory accounts not only for the minimal code of the `sleep` binary itself, but also for mapping shared C libraries (`libc.so`), the runtime dynamic linker (`ld-linux.so`), the stack, and kernel page table structures allocated to manage the process's virtual memory space.

Interesting Output Highlight (Part 3)

The comparison between `MemFree: 6081672 kB` (~6.08 GB) and `MemAvailable: 10608164 kB` (~10.6 GB) in `/proc/meminfo` demonstrates the efficiency of the kernel page cache. The OS keeps ~4.4 GB of previously read disk pages cached in RAM rather than letting the memory sit unused, accelerating future I/O while still guaranteeing that this RAM can be reclaimed instantly when needed.

Part 4. Devices and storage (I/O management)

Storage and Device Listing

```
❯ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p2  235G  145G   90G  62% /
devtmpfs        9.7G     0  9.7G   0% /dev
tmpfs           9.7G   56M  9.7G   1% /dev/shm
efivarfs        148K  140K  3.4K  98% /sys/firmware/efi/efivars
tmpfs           3.9G  2.1M  3.9G   1% /run
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
/dev/nvme0n1p2  235G  145G   90G  62% /home
/dev/nvme0n1p2  235G  145G   90G  62% /root
/dev/nvme0n1p2  235G  145G   90G  62% /srv
/dev/nvme0n1p2  235G  145G   90G  62% /var/cache
tmpfs           9.7G  1.2G  8.6G  12% /tmp
/dev/nvme0n1p2  235G  145G   90G  62% /var/tmp
/dev/nvme0n1p2  235G  145G   90G  62% /var/log
/dev/nvme0n1p1  4.0G  286M  3.8G   7% /boot
/dev/nvme1n1p1  469G  259G  186G  59% /mnt/storage
tmpfs           2.0G  118M  1.9G   6% /run/user/1000

❯ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
zram0       253:0    0  19.4G  0 disk [SWAP]
nvme0n1     259:0    0 238.5G  0 disk
├─nvme0n1p1 259:1    0     4G  0 part /boot
└─nvme0n1p2 259:2    0 234.5G  0 part /var/log
                                      /var/tmp
                                      /var/cache
                                      /srv
                                      /root
                                      /home
                                      /
nvme1n1     259:3    0 476.9G  0 disk
└─nvme1n1p1 259:4    0 476.9G  0 part /mnt/storage

❯ du -sh /home/ilie/Documents/SO/os-lab1
12K /home/ilie/Documents/SO/os-lab1

❯ ls -l /dev | head
total 0
crw-r--r--   1 root root    10,   235 Sep 28 15:43 autofs
drwxr-xr-x   2 root root          160 Sep 28 15:43 block
crw-rw----   1 root disk    10,   234 Sep 28 15:43 btrfs-control
drwxr-xr-x   3 root root           60 Sep 27 16:18 bus
drwxr-xr-x   2 root root         4540 Sep 28 15:48 char
crw-------   1 root root     5,     1 Sep 28 15:43 console
```

```
lrwxrwxrwx   1 root root           11 Sep 27 16:18 core -> /proc/kcore
drwxr-xr-x   6 root root          120 Sep 28 15:27 cpu
crw-rw----   1 root audio   10,   259 Sep 28 15:43 cpu_dma_latency

❯ mount | head
/dev/nvme0n1p2 on / type btrfs
 ↪  (rw,noatime,compress=zstd:1,ssd,discard=async,space_cache=v2,subvolid=256,subvol=/@)
devtmpfs on /dev type devtmpfs (rw,nosuid,size=10078308k,nr_inodes=2519577,mode=755,inode64,huge=advise)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,huge=advise,usrquota)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
efivarfs on /sys/firmware/efi/efivars type efivarfs (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
```

Figure 4.1 – Storage topology, block devices, and filesystem mount points

Observe (Part 4 Answers)

1. **Which device is your root filesystem "/" mounted on?**
   - It is mounted on `/dev/nvme0n1p2` (partition 2 of the primary NVMe solid-state drive `nvme0n1`, using the `btrfs` filesystem with zstd compression).
2. **Give one entry from `/dev` and say what real thing it stands for.**
   - `/dev/nvme0n1` represents the physical NVMe solid-state drive (a block storage hardware device), while `/dev/console` represents the primary system console character device used to handle system output and interactive input.
3. **In one sentence: what does "everything is a file" mean, based on what you saw?**
   - "Everything is a file" means that the operating system abstracts diverse physical hardware devices, inter-process communication channels, and kernel runtime state through a single unified filesystem namespace, enabling programs to access storage disks, memory data, and peripheral devices using uniform file operations such as `open`, `read`, and `write`.

Interesting Output Highlight (Part 4)

In `ls -l /dev`, instead of displaying regular file sizes in bytes, entries display two numbers separated by a comma (for example, `10, 235` for `autofs` or `5, 1` for `console`). These correspond to the **major and minor device numbers**, where the major number identifies the kernel driver assigned to manage the device, and the minor number specifies the unique physical or logical instance of that device.

Conclusions

The operating system acts as the foundational resource manager of a computer by coordinating four core hardware resources: files and storage, processes, physical memory, and I/O devices.

We observed file management through commands like `ls -l` and `chmod`, which revealed how the OS translates raw storage into a structured tree governed by access permissions, while process management was demonstrated using `ps` and `/proc/<pid>/status` as the kernel created, isolated, scheduled, and tracked running tasks.

Finally, memory management was made visible with `free -h` and `meminfo` as the kernel balanced physical RAM allocations against dynamic caches and swap, while `lsblk` and `/dev` showed device management in action by exposing disparate hardware components through uniform file-like interfaces.